MediFlow — Comprehensive Viva Questions & Answers

Technical Reference & Project Defense Guide | Next.js 15, PostgreSQL (Supabase), Prisma ORM

> **Target Project:** MediFlow (Smart Healthcare Access & Health Record System)
> **Stack:** Next.js 15 (App Router), TypeScript, Tailwind CSS, PostgreSQL on Supabase, Prisma ORM, PgBouncer
> **Target Audience:** Technical Project Viva, Examination Panels, and Code Review Defenses

---

Category 1: Project Overview & Business Logic

Q1: What is the core problem MediFlow solves, and what is its unique selling proposition (USP)?

Answer: MediFlow solves two major healthcare issues:

- **Hospital Overcrowding:** Through real-time crowd level monitoring (`LOW` , `MODERATE` , `HIGH` , `VERY_HIGH`) and estimated waiting times.
- **Fragmented Medical Histories:** By linking appointments, diagnostic notes, digital prescriptions, and uploaded lab documents into a unified, chronological **Health Timeline**.

USP: Real-time hospital crowd visibility combined with double-booking-protected appointment scheduling and doctor digital prescription generation.

Q2: How does the double-booking prevention mechanism work?

Answer: In `src/app/api/appointments/route.ts` , before confirming an appointment:

- The API queries the database with an indexed compound filter: `(doctorId, date, timeSlot)` .
- It checks if any booking exists with the status `CONFIRMED` or `REQUESTED` .
- If found, the server rejects the request immediately with an **HTTP 409 Conflict** and the error code `SLOT_ALREADY_BOOKED` .

Category 2: Frontend & Next.js Architecture

Q3: What is the difference between Server Components and Client Components in your application?

Answer:

- **Server Components (Default in Next.js):** Render on the server, fetch data directly without exposing API keys, and ship **0 kB of JavaScript** to the browser (e.g., Landing page, static layouts).
- **Client Components (marked with `"use client"`):** Used whenever user interactivity, browser hooks (`useState` , `useEffect`), or event listeners (`onClick` , `onChange`) are needed (e.g., booking form, filters, charts).

Q4: What is the role of layout.tsx versus page.tsx?

Answer:

- `layout.tsx` : Defines the shared UI (like the Navbar, Sidebar, font configuration, and background styling) that persists across page navigations without re-rendering.
- `page.tsx` : Defines the unique UI and content for a specific route segment (e.g., `/patient/appointments` or `/doctor/prescriptions`).

Q5: Why did you use Tailwind CSS instead of traditional CSS or Bootstrap?

Answer: Tailwind CSS is a utility-first CSS framework. It avoids writing bloated custom CSS files, ensures design consistency, and automatically purges unused styles during the production build (`npm run build`), resulting in a tiny CSS footprint (~10–20 kB).

Category 3: Backend & API Design

Q6: Where is your backend server hosted, and why don't you have a separate Express.js server?

Answer: Next.js 15 has built-in **Route Handlers** (files named `route.ts` inside `src/app/api/`). Each handler functions as a serverless microservice executing HTTP methods (`GET`, `POST`, `PATCH`, `DELETE`). This eliminates the maintenance, hosting cost, and CORS issues of maintaining a separate Express server.

Q7: What is the purpose of src/middleware.ts?

Answer: The Middleware acts as a gatekeeper at the network edge. It intercepts every incoming HTTP request before it reaches a page:

- It verifies the user's JWT session cookie.
- If a Patient tries to access `/admin` or `/doctor`, the middleware instantly redirects them to `/patient`.
- It bypasses static assets (images, CSS, JS) using regex matchers so it adds zero latency to media requests.

Q8: Why did you use Zod for validation?

Answer: Zod provides schema validation with static type inference. Before writing booking or prescription data to the database, Zod verifies that required fields are present, dates are correctly formatted, and string lengths are within safe bounds, rejecting bad requests with HTTP 400.

Category 4: Database & Prisma ORM

Q9: What is Prisma ORM, and how does it interact with Supabase?

Answer: Prisma is a modern TypeScript ORM (Object-Relational Mapping). It defines database tables in `prisma/schema.prisma`. When we run `npx prisma generate`, Prisma creates a strongly typed client with full auto-completion for querying PostgreSQL without writing raw SQL strings.

Q10: What is the difference between npx prisma db push and npx prisma migrate dev?

Answer:

- `prisma db push`: Directly synchronizes the Prisma schema with the database without creating SQL migration history files. It is faster during active prototyping and ideal for cloud databases like Supabase.
- `prisma migrate dev`: Generates ordered SQL migration files (`.sql`) to track incremental database changes over time in team environments.

Q11: Explain the key relationships in your database schema.

Answer:

- **One-to-One:** `User` ↔ `PatientProfile` (One user account has exactly one patient profile).
- **One-to-Many:** `DoctorProfile` ↔ `Appointment` (One doctor has many appointments).
- **Many-to-Many:** `DoctorProfile` ↔ `Hospital` (Doctors work across multiple hospitals via the `DoctorHospital` join table).
- **Cascade Delete:** When a `User` is deleted, their associated `PatientProfile` or `DoctorProfile` is automatically deleted (`onDelete: Cascade`).

Q12: What is Connection Pooling (PgBouncer), and why is it necessary?

Answer: PostgreSQL allocates a separate OS process/memory for every client connection. In serverless deployments (like Vercel), hundreds of function instances can spin up simultaneously, easily exhausting PostgreSQL's maximum connection limit. **PgBouncer (port 6543)** acts as a pooler, reusing a fixed number of open database connections across multiple incoming serverless requests.

Category 5: Authentication & Security

Q13: Why did you store JWTs in HttpOnly Cookies instead of localStorage?

Answer: Storing tokens in `localStorage` leaves them vulnerable to Cross-Site Scripting (XSS) attacks because any injected JavaScript script can read `localStorage.getItem('token')`. An **HttpOnly** cookie cannot be accessed by client-side JavaScript, completely protecting the session token from token theft.

Q14: What is the difference between Hashing and Encryption? Which one did you use for passwords?

Answer:

- **Encryption:** A two-way process where cipher-text can be decrypted back into plain text using a key.
- **Hashing:** A one-way mathematical function where the output cannot be reversed into plain text.

We used **Hashing (`bcryptjs`) with 10 salt rounds**. Even if the database is leaked, the plain text passwords cannot be recovered.

Category 6: Performance & Optimization (Examiner Impressers)

Q15: How did you optimize database latency when connecting to cloud Supabase?

Answer:

- **Query Parallelization:** Replaced sequential database requests with `Promise.all` (e.g., querying appointments, doctors, and notifications concurrently), reducing multi-query wait time from ~10.4s to ~1.8s (an **82.3% latency reduction**).
- **Global Client Singleton:** Cached the Prisma client on `globalThis` in `src/lib/db.ts` to prevent repeated TLS handshakes.
- **statement_cache_size=0:** Configured on the connection string to prevent prepared-statement conflicts with PgBouncer's transaction mode.

Q16: How did you optimize the frontend bundle size?

Answer: In `next.config.ts`, we enabled `optimizePackageImports` for heavy libraries like `lucide-react`, `date-fns`, and `recharts`. This enables tree-shaking so only the imported icons/functions are bundled, keeping the initial shared JavaScript payload down to just **102 kB**.
